10_s(s+1)

October 30, 2026

[155]:
```
import numpy as np
import matplotlib.pyplot as plt
import control as ct
import scipy.signal as signal
import math
```

[156]:
```
K = 10
G1 = ct.tf([1], [1, 0])
G2 = ct.tf([1], [1, 1])

# Gain of the system K
print(f'The gain of the system K: {K}')

# Actual Plant
G = K * G1 * G2
print(f'The transfer function of the system G(s): {G}')

# Feedback Loop Transfer Function
H = 1
print(f'The transfer function of the feedback loop H(s): {H}')

# Open Loop Transfer Funtion
L = G * H
print(f'The transfer function of the open loop system L(s): {L}')

# Closed Loop Transfer Funtion
CL = ct.feedback(G, H)
print(f'The transfer function of the closed loop system CL(s): {CL}')
```

```
The gain of the system K: 10
The transfer function of the system G(s): <TransferFunction>: sys[261]
Inputs (1): ['u[0]']
Outputs (1): ['y[0]']

    10
  -------
  s^2 + s
The transfer function of the feedback loop H(s): 1
```

```
The transfer function of the open loop system L(s): <TransferFunction>: sys[263]
Inputs (1): ['u[0]']
Outputs (1): ['y[0]']

    10
  -------
  s^2 + s
The transfer function of the closed loop system CL(s): <TransferFunction>:
sys[265]
Inputs (1): ['u[0]']
Outputs (1): ['y[0]']

       10
  ------------
  s^2 + s + 10
```

[157]:
```python
# Plot the Pole-Zero map with the damping/frequency grid
ct.pole_zero_plot(L, grid=True, marker_size=10, marker_width=3)
plt.title("Pole-Zero Map of OL System")
plt.show()

# Root locus = drawn for open loop system to see closed loop poles location at␣
 ↪s plane
ct.root_locus(L, grid=True, marker_size=10, marker_width=3)
plt.show()

# DC gain
dc = ct.dcgain(L)
print(f'DC gain for OL= {dc}')
# Zeros
z = ct.zeros(L)
print(f'Zeros of OL= {z}')
# Poles
p = ct.poles(L)
print(f'Poles of OL= {p}')

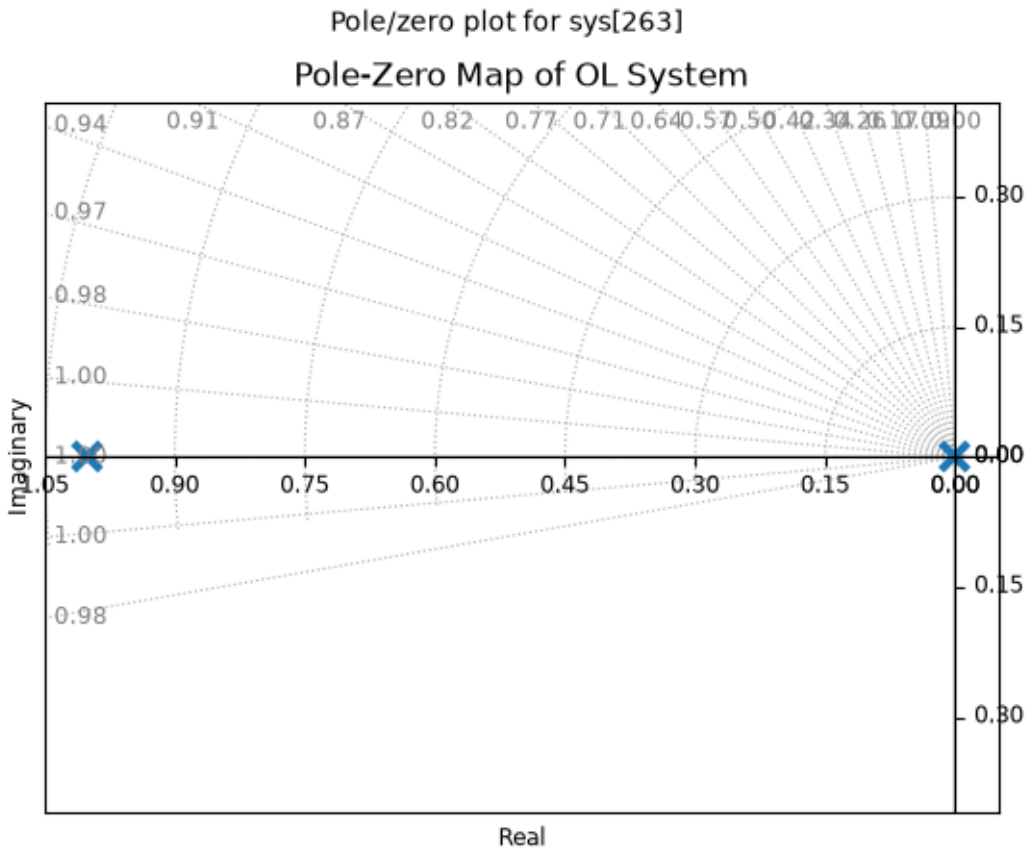
tol = 1e-8
# keep only strictly stable poles (real part < 0)
stable_mask = np.real(p) < -tol

if np.any(stable_mask):
    p_stable = p[stable_mask]
    rp = np.real(p_stable)
    max_rp = np.max(rp)
    dominant_poles = p_stable[np.isclose(rp, max_rp, atol=tol)]
else:
    dominant_poles = np.array([])
```

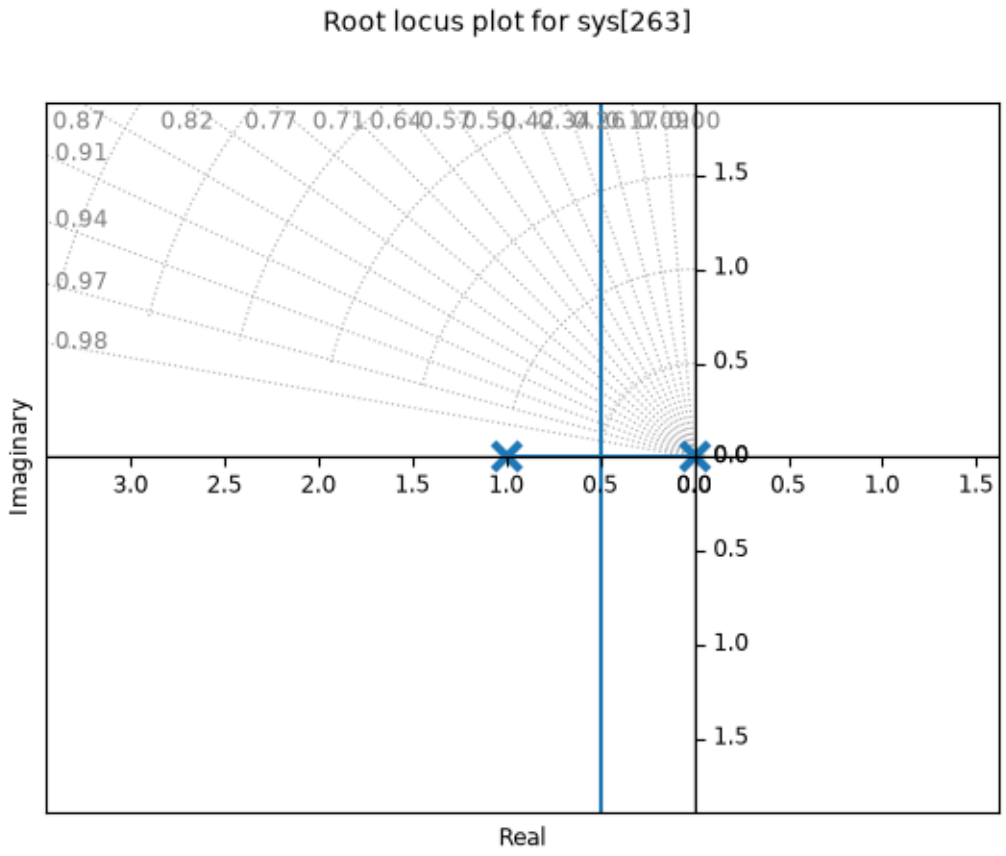
```
    print("No strictly stable pole -> no classical dominant pole.")

print("Dominant stable OL pole(s):", dominant_poles)

p1 = dominant_poles[0]
print(f'Dominant OL pole (p1) = {p1}')
print(f'Dominant OL pole (p1) in abs value = {np.abs(p1)}')
```




```
Ignoring fixed x limits to fulfill fixed data aspect with adjustable data
limits.
```




```
DC gain for OL= inf
Zeros of OL= []
Poles of OL= [-1.+0.j  0.+0.j]
Dominant stable OL pole(s): [-1.+0.j]
Dominant OL pole (p1) = (-1+0j)
Dominant OL pole (p1) in abs value = 1.0
```

[158]:
```python
# closed loop poles and zeros analysis

# Plot the Pole-Zero map with the damping/frequency grid
ct.pole_zero_plot(CL, grid=True, marker_size=10, marker_width=3)
plt.title("Pole-Zero Map of CL System")
plt.show()

# DC gain
dc = ct.dcgain(CL)
print(f'DC gain for CL= {dc}')
# Zeros
z = ct.zeros(CL)
print(f'Zeros of CL= {z}')
```

```
# Poles
p = ct.poles(CL)
print(f'Poles of CL= {p}')

# Print exact transient metrics derived from poles
wn, zeta, poles = ct.damp(CL)
print(f'Natural frequency of poles (Wn) = {wn}')
print(f'Damping ratio zeta of poles (Z) = {zeta}')
print(f'Poles = {poles}')

# To find the dominant pole(s) of a control system plant,
# you identify the poles (the roots of the denominator of the transfer␣
 ↪function) that lie closest to the imaginary (jw) axis
# or the origin in the s-plane,
# provided other poles are sufficiently far away
print(f'Poles of CL= {p}')

tol = 1e-8
# keep only strictly stable poles (real part < 0)
stable_mask = np.real(p) < -tol

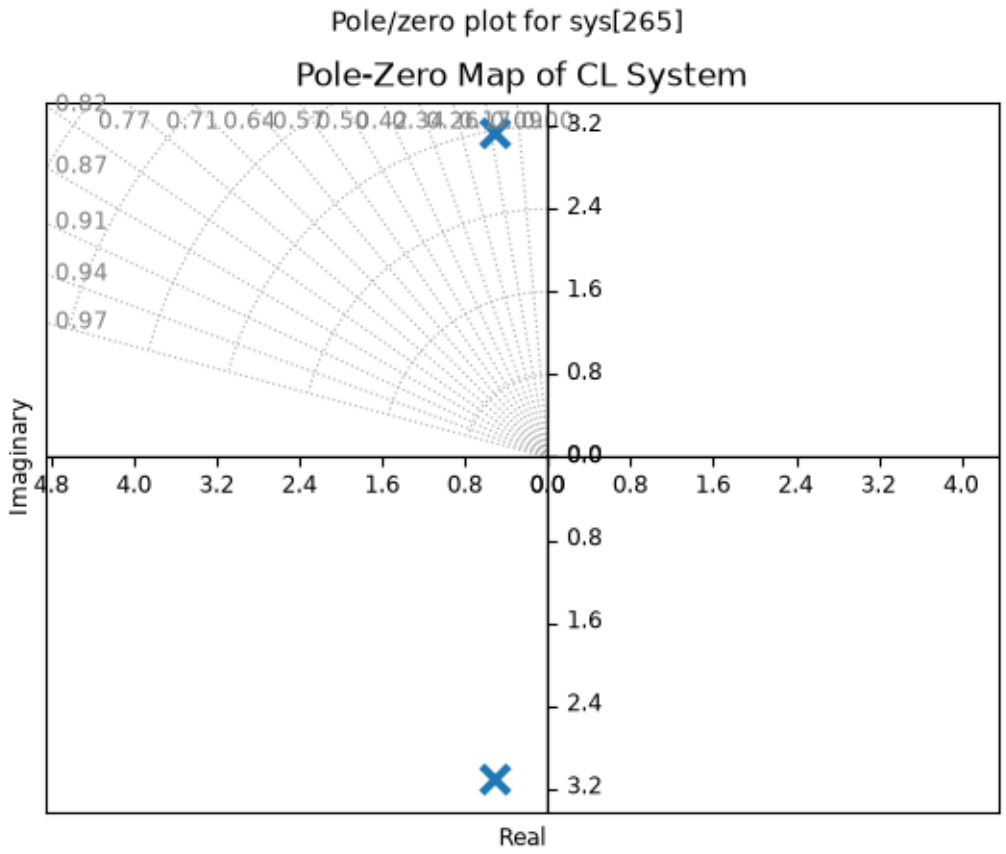
if np.any(stable_mask):
    p_stable = p[stable_mask]
    rp = np.real(p_stable)
    max_rp = np.max(rp)
    dominant_poles = p_stable[np.isclose(rp, max_rp, atol=tol)]
else:
    dominant_poles = np.array([])
    print("No strictly stable pole -> no classical dominant pole.")

print("Dominant stable CL pole(s):", dominant_poles)

p1 = dominant_poles[0]
print(f'Dominant CL pole (p1) = {p1}')
print(f'Dominant CL pole (p1) in abs value = {np.abs(p1)}')
print('\n')

wn = abs(p1)                    # Natural frequency (rad/s)
zeta = -np.real(p1) / wn      # Damping ratio
sigma = -np.real(p1)          # Decay rate (1/s)
wd = abs(np.imag(p1))         # Damped frequency (rad/s)

print("Dominant Pole:", p1)
print("Natural frequency (wn):", wn, "rad/s")
print("Damping ratio (zeta, z):", zeta)
print("Decay rate (sigma):", sigma, "1/s")
print("Damped frequency (wd):", wd, "rad/s")
```




```
DC gain for CL= 1.0
Zeros of CL= []
Poles of CL= [-0.5+3.122499j -0.5-3.122499j]
    Eigenvalue (pole)       Damping     Frequency
      -0.5    +3.122j        0.1581         3.162
      -0.5    -3.122j        0.1581         3.162
Natural frequency of poles (Wn) = [3.16227766 3.16227766]
Damping ratio zeta of poles (Z) = [0.15811388 0.15811388]
Poles = [-0.5+3.122499j -0.5-3.122499j]
Poles of CL= [-0.5+3.122499j -0.5-3.122499j]
Dominant stable CL pole(s): [-0.5+3.122499j -0.5-3.122499j]
Dominant CL pole (p1) = (-0.49999999999999994+3.1224989991991987j)
Dominant CL pole (p1) in abs value = 3.162277660168379

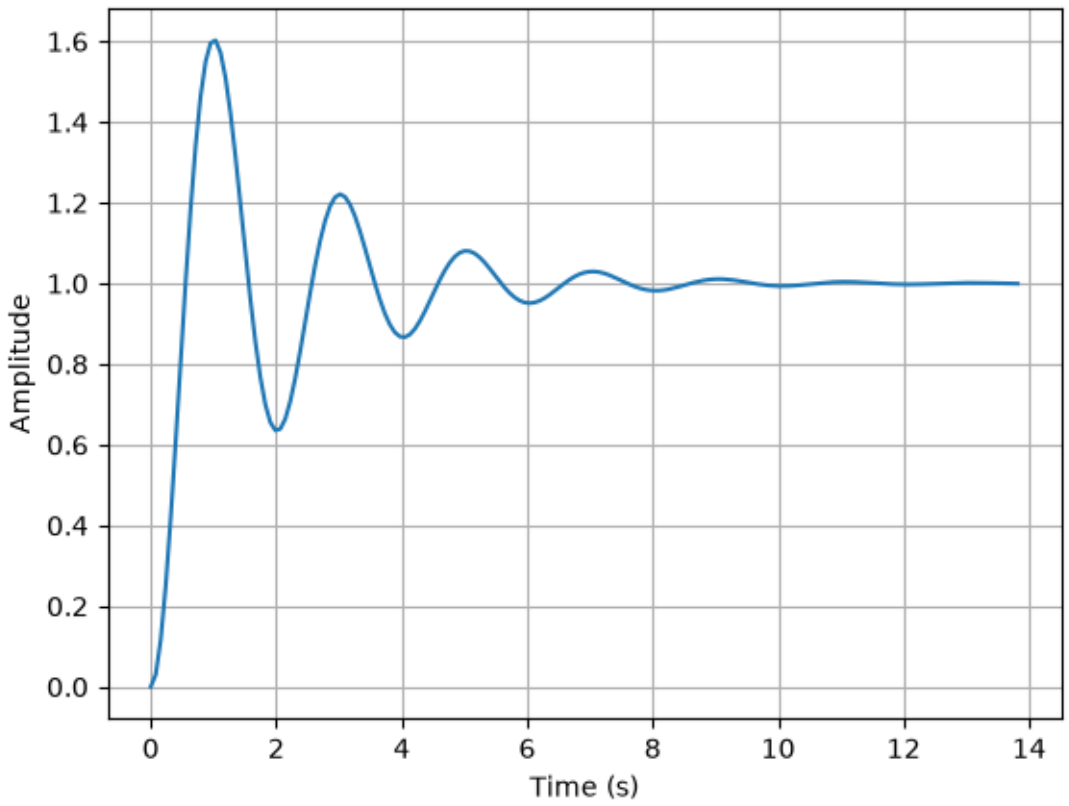

Dominant Pole: (-0.49999999999999994+3.1224989991991987j)
Natural frequency (wn): 3.162277660168379 rad/s
Damping ratio (zeta, z): 0.15811388300841897
Decay rate (sigma): 0.49999999999999994 1/s
Damped frequency (wd): 3.1224989991991987 rad/s
```

[159]:
```
plt.figure()
t, y = ct.step_response(CL)

plt.plot(t, y)
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.grid(True)
plt.show()

# transient characteristics
info = ct.step_info(CL)
final_value = info['SteadyStateValue']
ess = 1 - final_value


print(f"Rise Time (Tr): {info['RiseTime']:.2f} seconds")
print(f"Settling Time (Ts): {info['SettlingTime']:.2f} seconds")
print(f"Peak Time (Tp): {info['PeakTime']:.2f} seconds")
print(f"Overshoot (%): {info['Overshoot']:.2f}%")
# print(f"Undershoot (%): {info['Undershoot']:.2f}%")
print(f'Final Value = {info["SteadyStateValue"]}')
print(f'Steasy state error (ess) = {ess}')
```



```
Rise Time (Tr): 0.40 seconds
Settling Time (Ts): 7.38 seconds
Peak Time (Tp): 1.03 seconds
Overshoot (%): 60.26%
Final Value = 1.0
Steasy state error (ess) = 0.0
```

[160]:
```
# Required Specification

# Settling time (Ts) <= 2 seconds
# Maximum Overshoot (Mp) <= 5%

# Ts = 4/(zeta * wn) => wn = 4/(zeta * Ts)
# Mp = exp((-zeta * pi)/sqrt(1 - zeta^2)) => zeta = sqrt((ln(Mp))^2 /␣
↪((ln(Mp))^2 + pi^2))
```

[161]:
```
Ts = 2
Mp = 0.05

# Maximum overshoot (Mp) = exp((-zeta * pi)/sqrt(1 - zeta^2))
# zeta = sqrt((ln(Mp))^2 / ((ln(Mp))^2 + pi^2))
zeta = np.sqrt((np.log(Mp))**2 / ((np.log(Mp))**2 + np.pi**2))

# Settling Time (Ts) ~ 4/(zeta * wn)
wn = 4/(zeta * Ts)
wd = wn * np.sqrt(1 - zeta**2)

# s = a + bj
a = -zeta * wn
b = wd
Sd = a + b*1j

print(f'Required Damping Ratio (zeta) = {zeta}')
print(f'Required Natural Frequency (wn) = {wn}')
print(f'Required Damped Frequency (wd) = {wd}')
print(f'Required Dominant Pole (Sd) = {Sd}')
```

```
Required Damping Ratio (zeta) = 0.6901067305598216
Required Natural Frequency (wn) = 2.898102440440741
Required Damped Frequency (wd) = 2.0973787820249776
Required Dominant Pole (Sd) = (-2+2.0973787820249776j)
```

[162]:
```
# # Let zeta = 0.5
# # Natural Frequency (wn) = 3 rad/s
# zeta = 0.5
# wn = 3
```

```
# Sd = -(zeta*wn)+1j*wn*np.sqrt(1-zeta**2)
# print(f'Required Dominant Pole (Sd) = {Sd}')
# print(f'Required Dominant Pole in absolute form (Sd) = {np.abs(Sd)}')
```

[163]:
```
# So alone Gain Adujustment cannot alone achieve the desired performance
# as real part of dominant pole = -3 and root locus can only provide real part␣
 ↪of poles = -0.5
```

[164]:
```
# Angle dificiency = 180 - angle(zeros) + angle(poles)
phi = np.rad2deg(np.angle(G(Sd))) - 180
print(f'Angle Deficiency phi = {phi}')

# Deficit angle must be contributed by a lead compensator.
phi_c = -phi
print(f'Angle that must be contributed by lead compensator = {phi_c}')
```

```
Angle Deficiency phi = -69.12969202214077
Angle that must be contributed by lead compensator = 69.12969202214077
```

[165]:
```
# Choose zero of the lead compensator to cancle the real dominant pole (not␣
 ↪complex) of the open or closed loop system
zc = 1
print(f'Lead Compensator Zero (zc) = {zc}')

# So lead compensator pole must be at required dominant pole
# pc = np.abs(Sd)
# print(f'Lead Compensator Pole (pc) = {pc}')

# And also it comes from angle criteria i.e. angle(G * H) = 180
# angle(zeros) - angle(poles) = 180
# Or for lead compensator angle(zeros) - angle(poles) = angle deficiency

w = np.rad2deg(np.angle(Sd + zc)) - phi_c
print(f'for compensator angle(zeros) - angle deficiency (w) = {w}')
v = np.tan(np.deg2rad(w))
print(f'tan of tan of w (v) = {v}')

pc = (np.imag(Sd)/v) - np.real(Sd)
print(f'pole of lead compensator = {pc}')

# Lead Compensator transfer function without gain
Gc = ct.tf([1, zc], [1, pc])
print(f'Lead Compensator Transfer Function without gain (Gc) = {Gc}')

print(f'Open Loop Gain at required Pole Gc(Sd) = {Gc(Sd)}')
```

```
print(f'Open Loop Gain at required Pole in absolute form Gc(Sd) = {np.
 ↪abs(Gc(Sd))}')
print(f'Open Loop Phase at required Pole Gc(Sd) = {np.angle(Gc(Sd))}')
print(f'Open Loop Phase at required Pole in Degrees Gc(Sd) = {np.rad2deg(np.
 ↪angle(Gc(Sd)))}')
```

```
Lead Compensator Zero (zc) = 1
for compensator angle(zeros) - angle deficiency (w) = 46.36144189013905
tan of tan of w (v) = 1.0486893910124895
pole of lead compensator = 3.9999999999999987
Lead Compensator Transfer Function without gain (Gc) = <TransferFunction>:
sys[270]
Inputs (1): ['u[0]']
Outputs (1): ['y[0]']

  s + 1
  -----
  s + 4
Open Loop Gain at required Pole Gc(Sd) = (0.285629050654051+0.7491532358266174j)
Open Loop Gain at required Pole in absolute form Gc(Sd) = 0.8017571485973952
Open Loop Phase at required Pole Gc(Sd) = 1.206540736676013
Open Loop Phase at required Pole in Degrees Gc(Sd) = 69.12969202214076
```

[166]:
```
# Lead compensator gain
Kc = 1/(np.abs(G(Sd)*Gc(Sd)))
print(f'Lead Compensator Gain (Kc) = {Kc}')
```

```
Lead Compensator Gain (Kc) = 0.8398997755288575
```

[167]:
```
Gc1 = Kc * Gc
print(f'Lead Compensator Transfer Function with gain (Gc1) = {Gc1}')

print(f'Open Loop Gain at required Pole G(Sd) = {Gc(Sd)}')
print(f'Open Loop Gain at required Pole in absolute form G(Sd) = {np.
 ↪abs(Gc(Sd))}')
print(f'Open Loop Phase at required Pole G(Sd) = {np.angle(Gc(Sd))}')
print(f'Open Loop Phase at required Pole in Degrees G(Sd) = {np.rad2deg(np.
 ↪angle(Gc(Sd)))}')
```

```
Lead Compensator Transfer Function with gain (Gc1) = <TransferFunction>:
sys[272]
Inputs (1): ['u[0]']
Outputs (1): ['y[0]']

  0.8399 s + 0.8399
  -----------------
        s + 4
Open Loop Gain at required Pole G(Sd) = (0.285629050654051+0.7491532358266174j)
Open Loop Gain at required Pole in absolute form G(Sd) = 0.8017571485973952
```

```
Open Loop Phase at required Pole G(Sd) = 1.206540736676013
Open Loop Phase at required Pole in Degrees G(Sd) = 69.12969202214076
```

[168]:
```
A1 = K*G2*Gc
# Type of system = 1 i.e. pole at origin = 1
# so the velocity error constant, Kv = lim:s->0 sG(s)Gc(s)
kv = np.abs(A1(0))
print(f'velocity error constant kv = {kv}')
```

```
velocity error constant kv = 2.500000000000001
```

[169]:
```
A2 = G*Gc1
print(f'System Transfer Function with compensator A2(s) = {A2}')
A3 = A2 * H
print(f'open loop system transfer function with compensator A3(s) = {A3}')
A4 = ct.feedback(A2, 1)
print(f'Closed loop system transfer function with compensator A4(s) = {A4}')
```

```
System Transfer Function with compensator A2(s) = <TransferFunction>: sys[276]
Inputs (1): ['u[0]']
Outputs (1): ['y[0]']

   8.399 s + 8.399
  -----------------
  s^3 + 5 s^2 + 4 s
open loop system transfer function with compensator A3(s) = <TransferFunction>:
sys[278]
Inputs (1): ['u[0]']
Outputs (1): ['y[0]']

   8.399 s + 8.399
  -----------------
  s^3 + 5 s^2 + 4 s
Closed loop system transfer function with compensator A4(s) =
<TransferFunction>: sys[280]
Inputs (1): ['u[0]']
Outputs (1): ['y[0]']

        8.399 s + 8.399
  ----------------------------
  s^3 + 5 s^2 + 12.4 s + 8.399
```

[170]:
```
# Plot the Pole-Zero map with the damping/frequency grid
ct.pole_zero_plot(A3, grid=True, marker_size=10, marker_width=3)
plt.title("Pole-Zero Map of OL System")
plt.show()
```

```
# Root locus = drawn for open loop system to see closed loop poles location at␣
 ↪s plane
ct.root_locus(A3, grid=True, marker_size=10, marker_width=3)
plt.show()

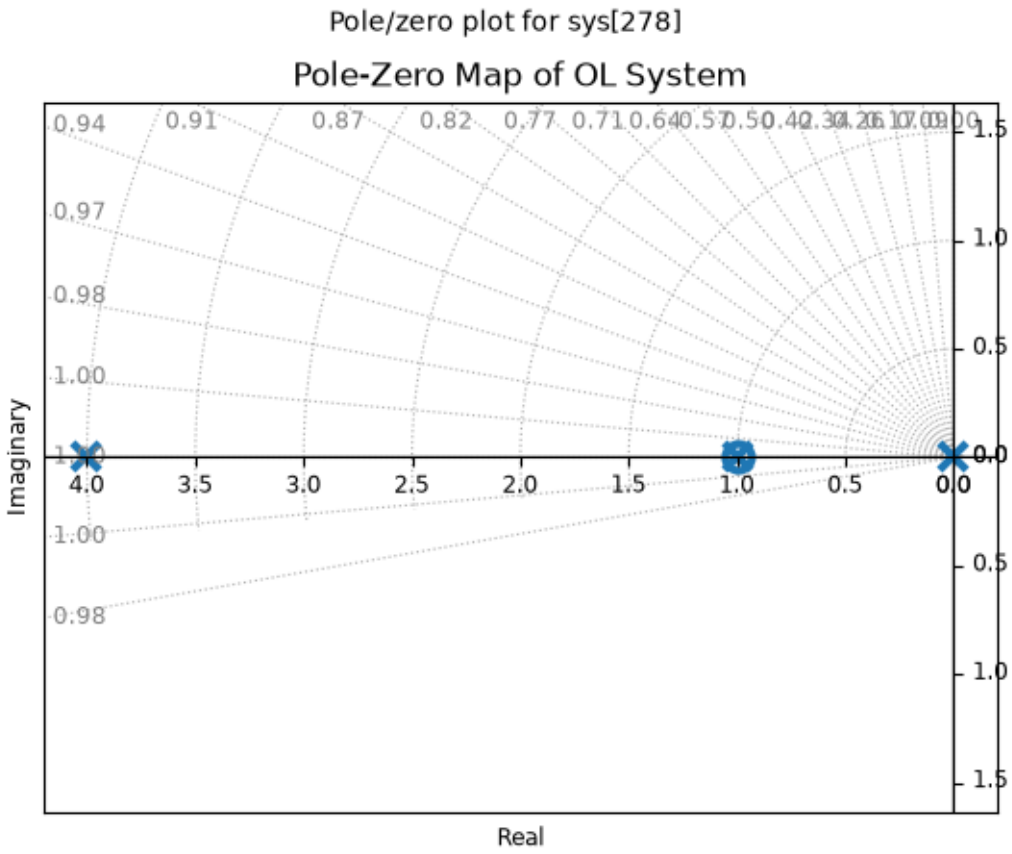
# DC gain
dc = ct.dcgain(A3)
print(f'DC gain for OL= {dc}')
# Zeros
z = ct.zeros(A3)
print(f'Zeros of OL= {z}')
# Poles
p = ct.poles(A3)
print(f'Poles of OL= {p}')

tol = 1e-8
# keep only strictly stable poles (real part < 0)
stable_mask = np.real(p) < -tol

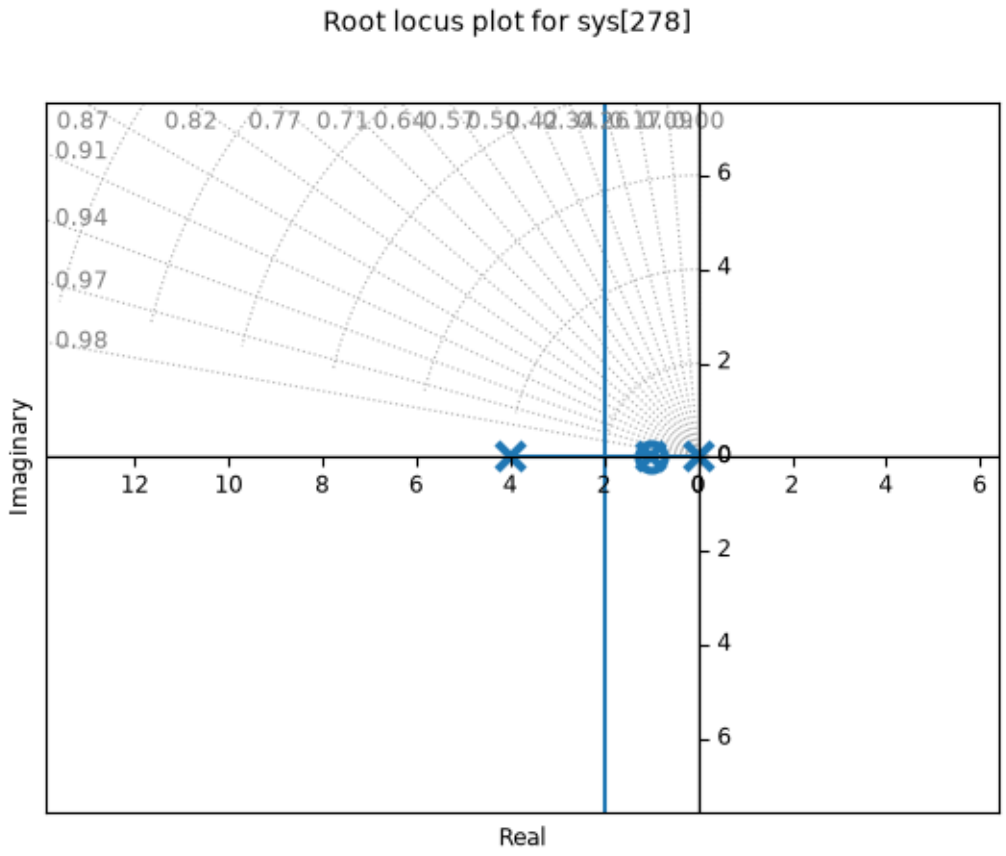
if np.any(stable_mask):
    p_stable = p[stable_mask]
    rp = np.real(p_stable)
    max_rp = np.max(rp)
    dominant_poles = p_stable[np.isclose(rp, max_rp, atol=tol)]
else:
    dominant_poles = np.array([])
    print("No strictly stable pole -> no classical dominant pole.")

print("Dominant stable OL pole(s):", dominant_poles)

p1 = dominant_poles[0]
print(f'Dominant OL pole (p1) = {p1}')
print(f'Dominant OL pole (p1) in abs value = {np.abs(p1)}')
```




```
Ignoring fixed x limits to fulfill fixed data aspect with adjustable data
limits.
```




```
DC gain for OL= inf
Zeros of OL= [-1.+0.j]
Poles of OL= [-4.+0.j -1.+0.j  0.+0.j]
Dominant stable OL pole(s): [-1.+0.j]
Dominant OL pole (p1) = (-1+0j)
Dominant OL pole (p1) in abs value = 1.0
```

[171]:
```python
# closed loop poles and zeros analysis

# Plot the Pole-Zero map with the damping/frequency grid
ct.pole_zero_plot(A4, grid=True, marker_size=10, marker_width=3)
plt.title("Pole-Zero Map of CL System")
plt.show()

# DC gain
dc = ct.dcgain(A4)
print(f'DC gain for CL= {dc}')
# Zeros
z = ct.zeros(A4)
print(f'Zeros of CL= {z}')
```

```
# Poles
p = ct.poles(A4)
print(f'Poles of CL= {p}')

# Print exact transient metrics derived from poles
wn, zeta, poles = ct.damp(A4)
print(f'Natural frequency of poles (Wn) = {wn}')
print(f'Damping ratio zeta of poles (Z) = {zeta}')
print(f'Poles = {poles}')

# To find the dominant pole(s) of a control system plant,
# you identify the poles (the roots of the denominator of the transfer␣
 ↪function) that lie closest to the imaginary (jw) axis
# or the origin in the s-plane,
# provided other poles are sufficiently far away
print(f'Poles of CL= {p}')

tol = 1e-8
# keep only strictly stable poles (real part < 0)
stable_mask = np.real(p) < -tol

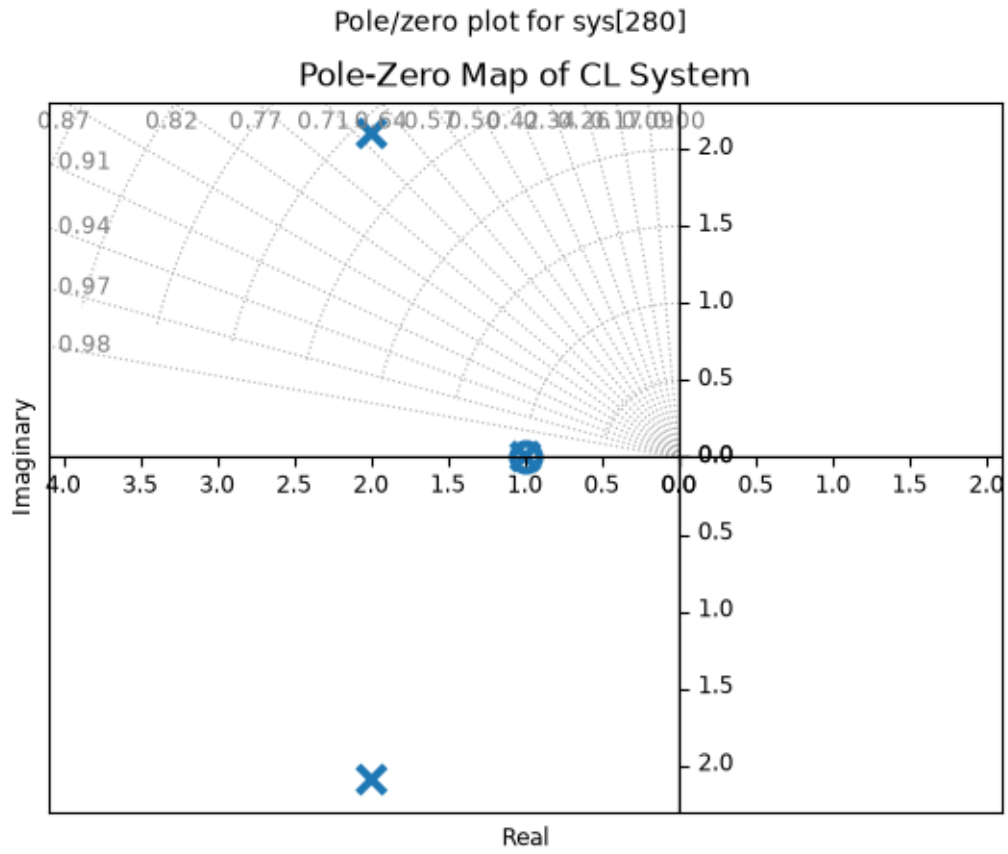
if np.any(stable_mask):
    p_stable = p[stable_mask]
    rp = np.real(p_stable)
    max_rp = np.max(rp)
    dominant_poles = p_stable[np.isclose(rp, max_rp, atol=tol)]
else:
    dominant_poles = np.array([])
    print("No strictly stable pole -> no classical dominant pole.")

print("Dominant stable CL pole(s):", dominant_poles)

p1 = dominant_poles[0]
print(f'Dominant CL pole (p1) = {p1}')
print(f'Dominant CL pole (p1) in abs value = {np.abs(p1)}')
print('\n')

wn = abs(p1)                    # Natural frequency (rad/s)
zeta = -np.real(p1) / wn      # Damping ratio
sigma = -np.real(p1)          # Decay rate (1/s)
wd = abs(np.imag(p1))         # Damped frequency (rad/s)

print("Dominant Pole:", p1)
print("Natural frequency (wn):", wn, "rad/s")
print("Damping ratio (zeta, z):", zeta)
print("Decay rate (sigma):", sigma, "1/s")
print("Damped frequency (wd):", wd, "rad/s")
```




```
DC gain for CL= 1.0
Zeros of CL= [-1.+0.j]
Poles of CL= [-2.+2.09737878j -2.-2.09737878j -1.+0.j        ]
    Eigenvalue (pole)       Damping     Frequency
        -2    +2.097j        0.6901         2.898
        -2    -2.097j        0.6901         2.898
                  -1             1             1
Natural frequency of poles (Wn) = [2.89810244 2.89810244 1.        ]
Damping ratio zeta of poles (Z) = [0.69010673 0.69010673 1.        ]
Poles = [-2.+2.09737878j -2.-2.09737878j -1.+0.j        ]
Poles of CL= [-2.+2.09737878j -2.-2.09737878j -1.+0.j        ]
Dominant stable CL pole(s): [-1.+0.j]
Dominant CL pole (p1) = (-1+0j)
Dominant CL pole (p1) in abs value = 1.0


Dominant Pole: (-1+0j)
Natural frequency (wn): 1.0 rad/s
Damping ratio (zeta, z): 1.0
Decay rate (sigma): 1.0 1/s
```

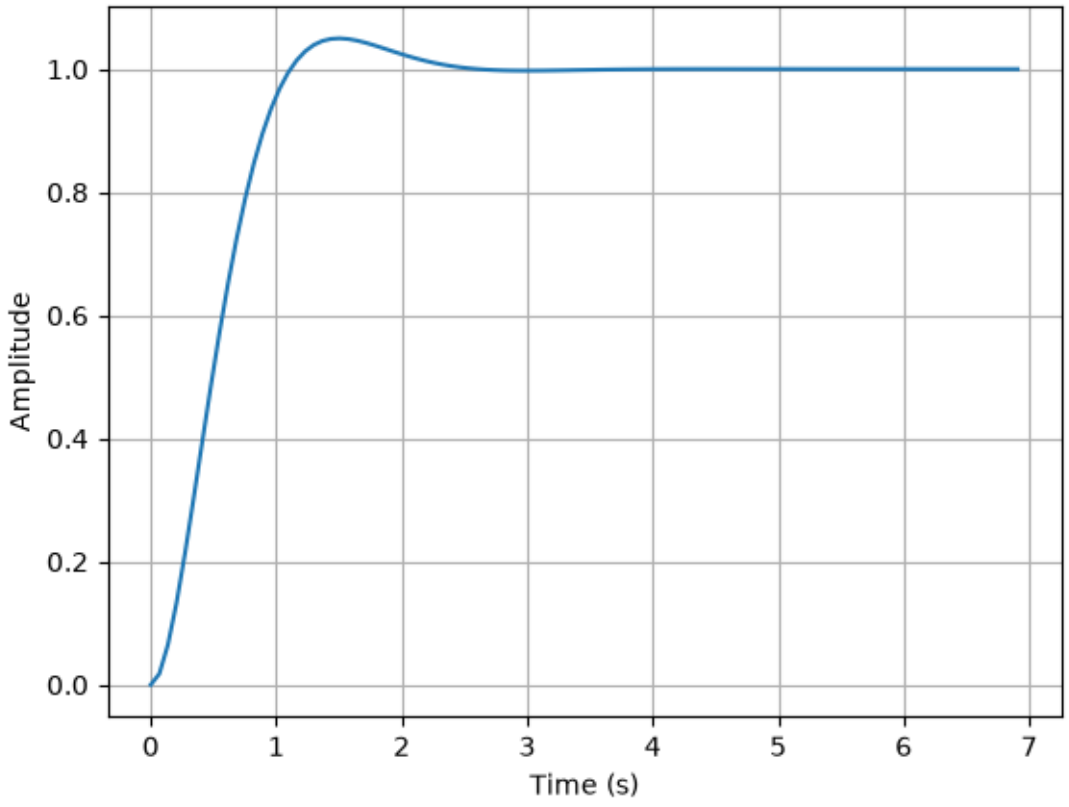
```
Damped frequency (wd): 0.0 rad/s
```

[172]:
```
plt.figure()
t, y = ct.step_response(A4)

plt.plot(t, y)
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.grid(True)
plt.show()

# transient characteristics
info = ct.step_info(A4)
final_value = info['SteadyStateValue']
ess = 1 - final_value

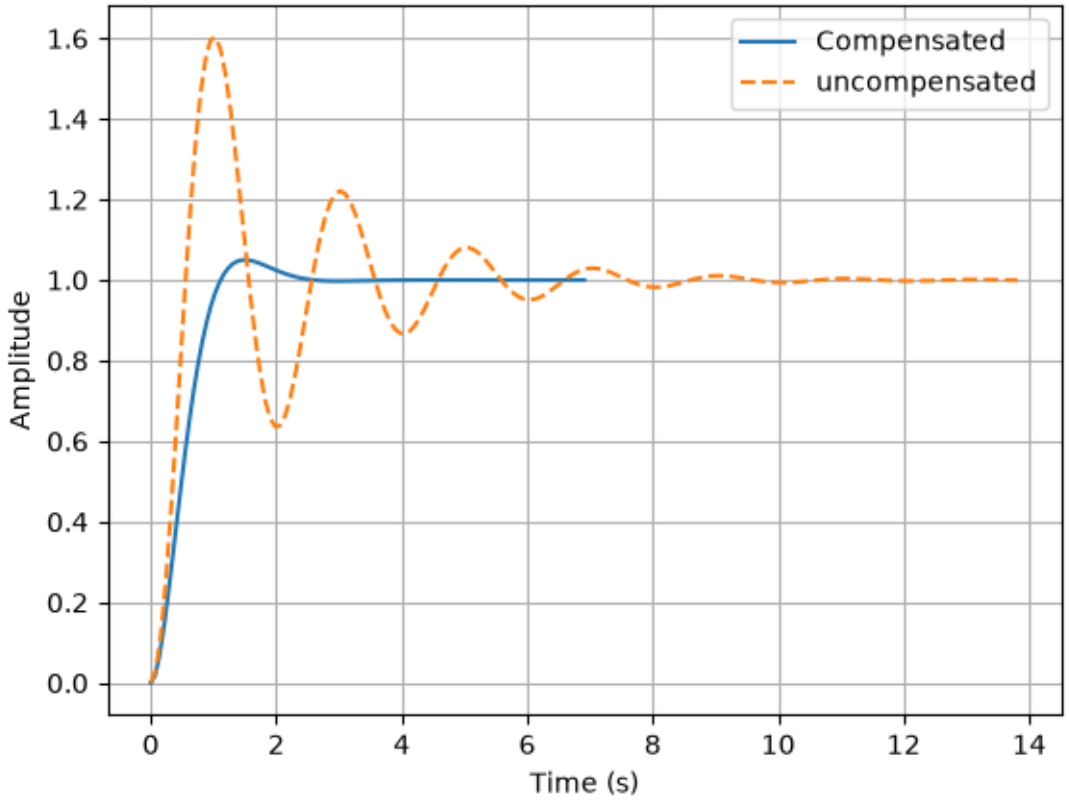

print(f"Rise Time (Tr): {info['RiseTime']:.2f} seconds")
print(f"Settling Time (Ts): {info['SettlingTime']:.2f} seconds")
print(f"Peak Time (Tp): {info['PeakTime']:.2f} seconds")
print(f"Overshoot (%): {info['Overshoot']:.2f}%")
# print(f"Undershoot (%): {info['Undershoot']:.2f}%")
print(f'Final Value = {info["SteadyStateValue"]}')
print(f'Steasy state error (ess) = {ess}')
```



```
Rise Time (Tr): 0.75 seconds
Settling Time (Ts): 2.12 seconds
Peak Time (Tp): 1.50 seconds
Overshoot (%): 5.00%
Final Value = 1.0
Steasy state error (ess) = 0.0
```

[173]:
```
plt.figure()
t1, y1 = ct.step_response(A4)
t2, y2 = ct.step_response(CL)

plt.plot(t1, y1, label='Compensated')
plt.plot(t2, y2, label='uncompensated', linestyle='--')
plt.xlabel('Time (s)')
plt.ylabel('Amplitude')
plt.grid(True)
plt.legend()
plt.show()
```



[ ]: